The Patching Landscape of Elisabeth-4 and the Mixed Filter Permutator Paradigm

Clément Hoffmann, Pierrick Méaux, François-Xavier Standaert

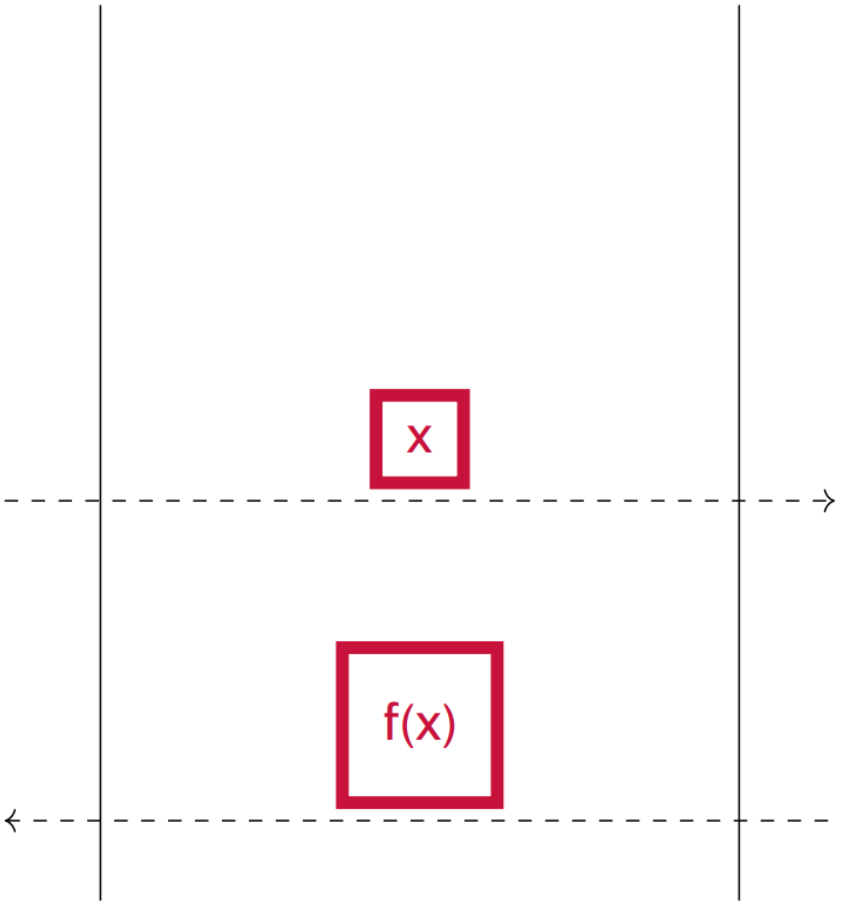
December 14, 2023





Hybrid Homomorphic Encryption (HHE)



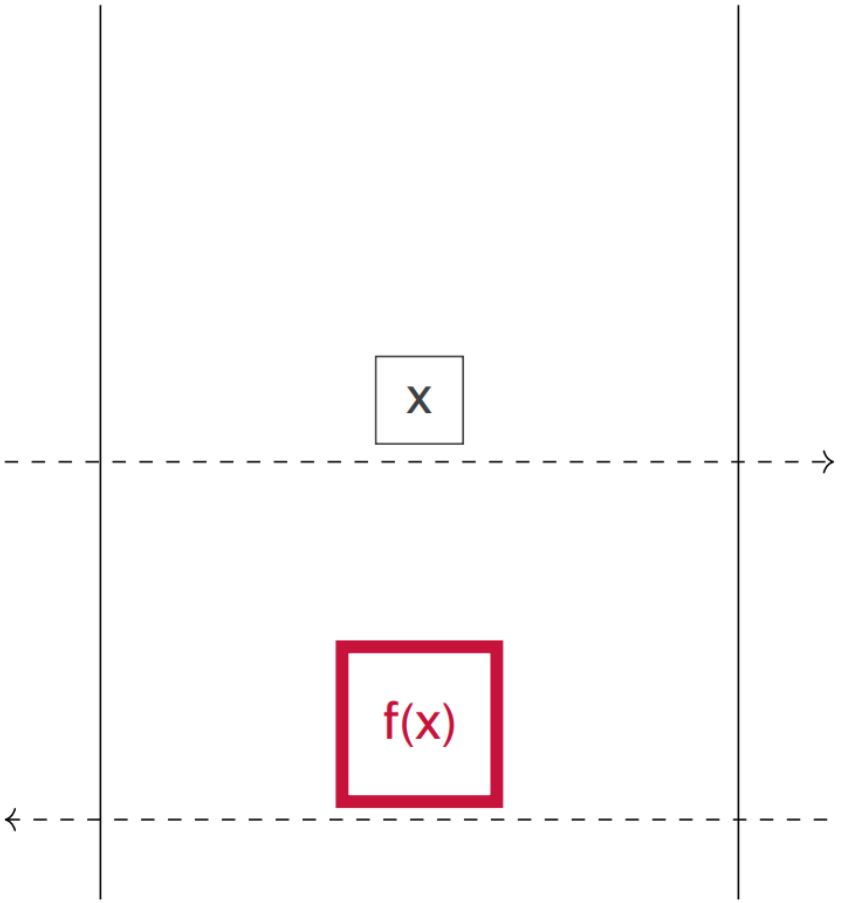




Homomorphically computes f(·)

Hybrid Homomorphic Encryption (HHE)





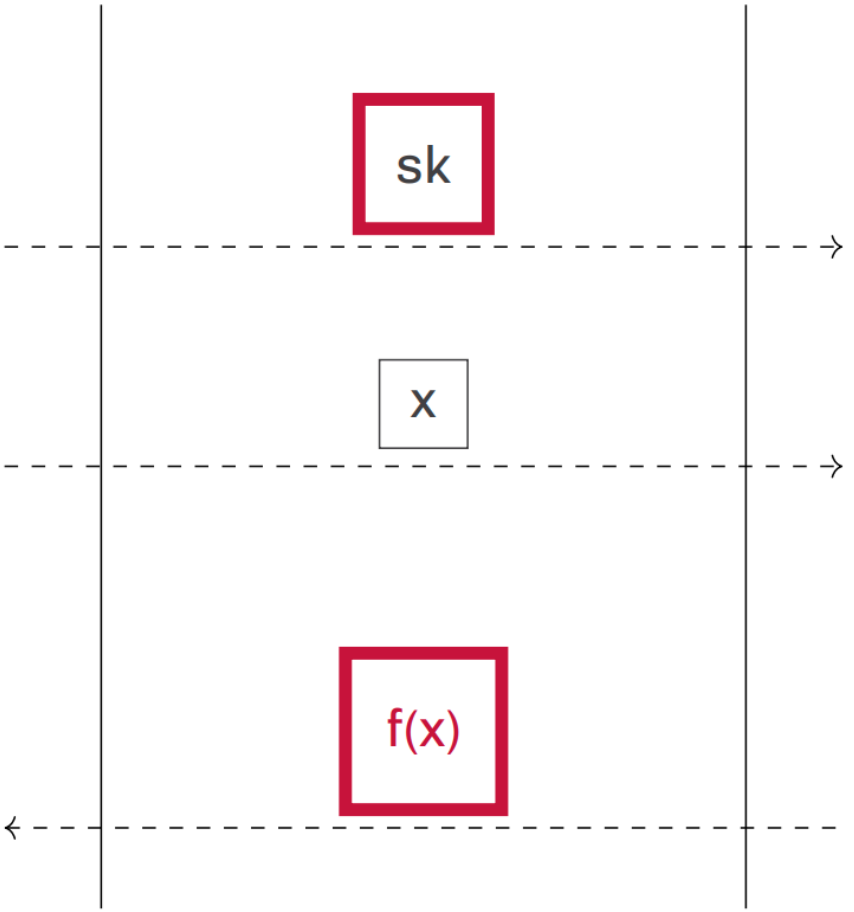


Homomorphically computes Dec(sk,·) to get x

Homomorphically computes f(·)

Hybrid Homomorphic Encryption (HHE)







Homomorphically computes $\mathsf{Dec}(sk,\cdot)$ to get x

Homomorphically computes $f(\cdot)$

HHE constraints on symmetric design



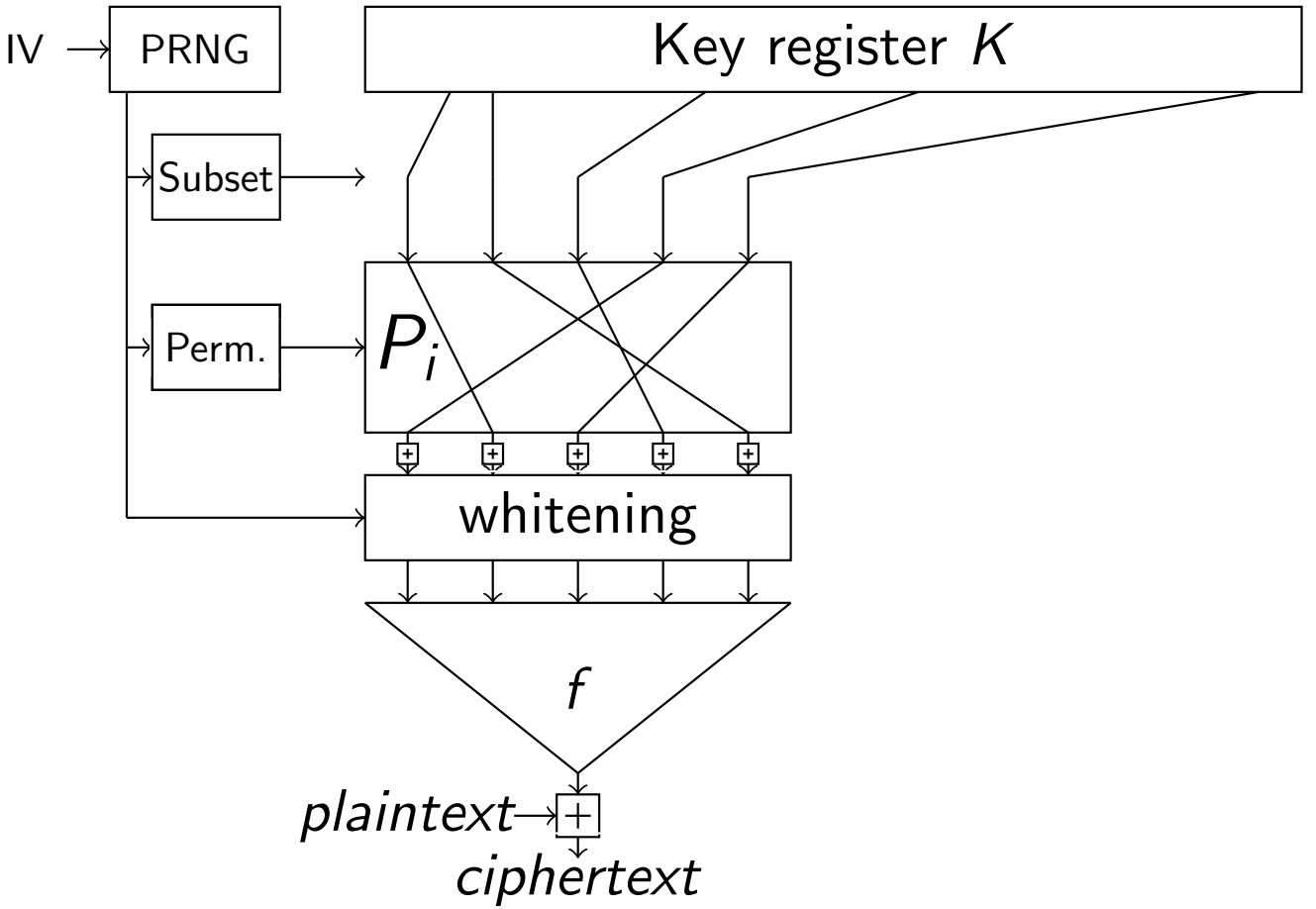
Efficient homomorphic evaluation = HE toolbox.

TFHE constraints:

- Addition modulo 2^k
- Negacyclic Look-up table on k bits

Group Filter Permutator (GFP) paradigm

Family of stream-ciphers, with a keystream in a group $\mathbb{G}$.



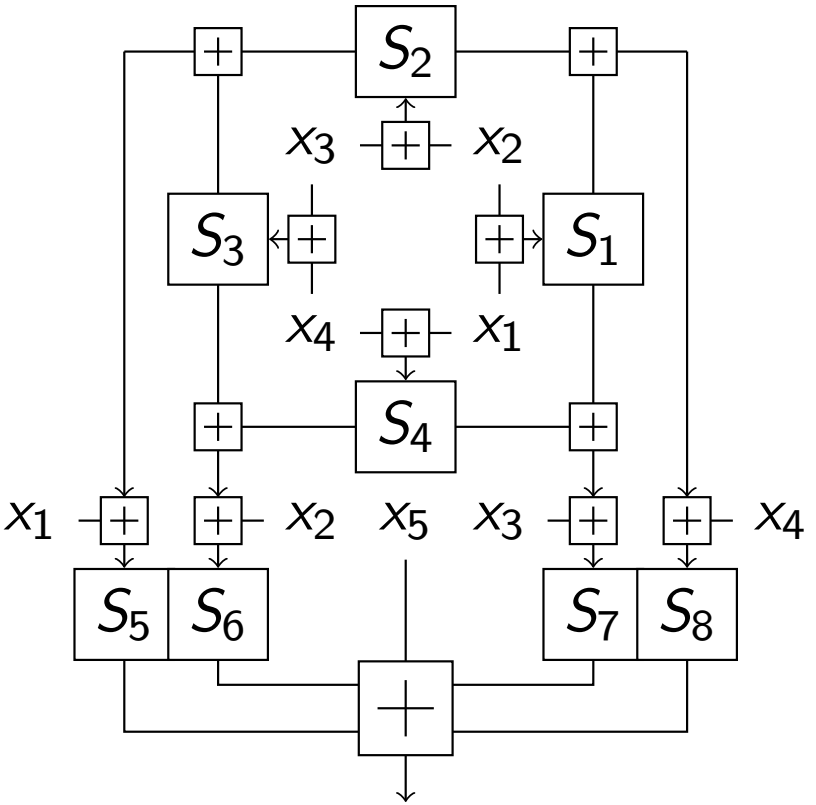
The Group Filter Permutator design

Elisabeth-4filter function



We compute the inner function characteristics and use them to get bound on the filter characteristics.

Elisabeth-4inner function



Elisabeth-4's 5-to-1 inner function.

Elisabeth-4security - Boolean function



On $\mathbb{F}_2$:

- Degree $\rightarrow$ Algebraic Immunity $\rightarrow$ Algebraic attacks
- Non linearity $\rightarrow$ Correlation attacks
- Guess and determine strategies

Elisabeth-4security - Boolean function



On $\mathbb{F}_2$:

- Degree $\rightarrow$ Algebraic Immunity $\rightarrow$ Algebraic attacks
- Non linearity $\rightarrow$ Correlation attacks
- Guess and determine strategies

On $\mathbb{Z}_{16}$:

We verify that the function is far from polynomials.

Linearization attack from [GHBJR23][1]



Vulnerability: **Low number of monomials** leads to a **linearization attack**.

[1] [GHBJR23] Gilbert, Heim Boissier, Jean, Reinhard. Cryptanalysis of elisabeth-4

Linearization attack from [GHBJR23][1]



Vulnerability: **Low number of monomials** leads to a **linearization attack**.

Why? Blocks only mix 4 key chunks. Key chunks from different blocks don't interact.

Other optimizations that improves this main idea.

[1] [GHBJR23] Gilbert, Heim Boissier, Jean, Reinhard. Cryptanalysis of elisabeth-4

Linearization attack from [GHBJR23][1]



Vulnerability: **Low number of monomials** leads to a **linearization attack**.

Why? Blocks only mix 4 key chunks. Key chunks from different blocks don't interact.

Other optimizations that improves this main idea.

Guaranteeing enough monomials for the filter function thwarts the attack

[1] [GHBJR23] Gilbert, Heim Boissier, Jean, Reinhard. Cryptanalysis of elisabeth-4

Outline



- How do we run the security analysis?

- 3 patches, with increasing level of modification from the original scheme:
  - Elisabeth-b4 modifies the inner function and increase parameter size.
  - Gabriel-4 mixes different inner functions to improve performances.
  - Margrethe-18-4 mixes two group representations to fix the issue raised by handling key chunks.

Security analysis details - $\mathbb{Z}_{16}$



[SHW23][2]: the probability for a random function $g : \mathbb{Z}_{16}^4 \longmapsto \mathbb{Z}_{16}$ to be a polynomial is $<< 2^{-128}$.

- We verify that the inner function is not a polynomial.
- We use a code-based argument to justify that the filter does not share many output with a low degree function.

[2]Specker, Hungerbühler, Wasem. The ring of polyfunctions over z/nz

Security analysis - Guess and determine



An adversary can set ℓ among n variables.

Security analysis - Guess and determine



An adversary can set ℓ among n variables.

Security analysis - Guess and determine



An adversary can set ℓ among n variables.

Cost: #Choice of variables $\times$ #Choice of values $= \binom{n}{\ell} \times 2^{\ell}$.

Security analysis - Guess and determine



An adversary can set ℓ among n variables.

Cost: #Choice of variables $\times$ #Choice of values $= \binom{n}{\ell} \times 2^{\ell}$.

Worth it if the degree or non linearity significantly decreases $\rightarrow$ we have to check for every possible guess.

Example:

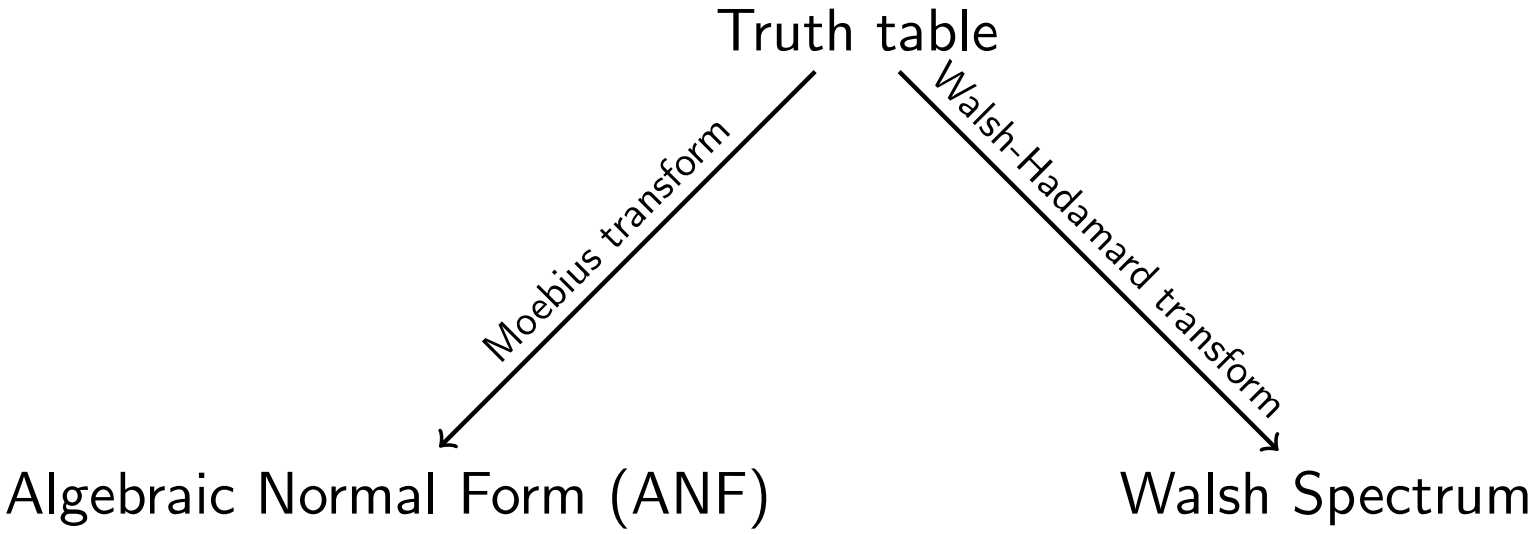
$$f : (x_0, x_1, x_2) \longmapsto x_0x_1x_2 + x_0x_2 + x_0 + x_1 + 1$$

Guess: $x_0 = 0 \rightarrow$ subfonction $f' = x_1 + 1$

Security analysis details over $\mathbb{F}_2$

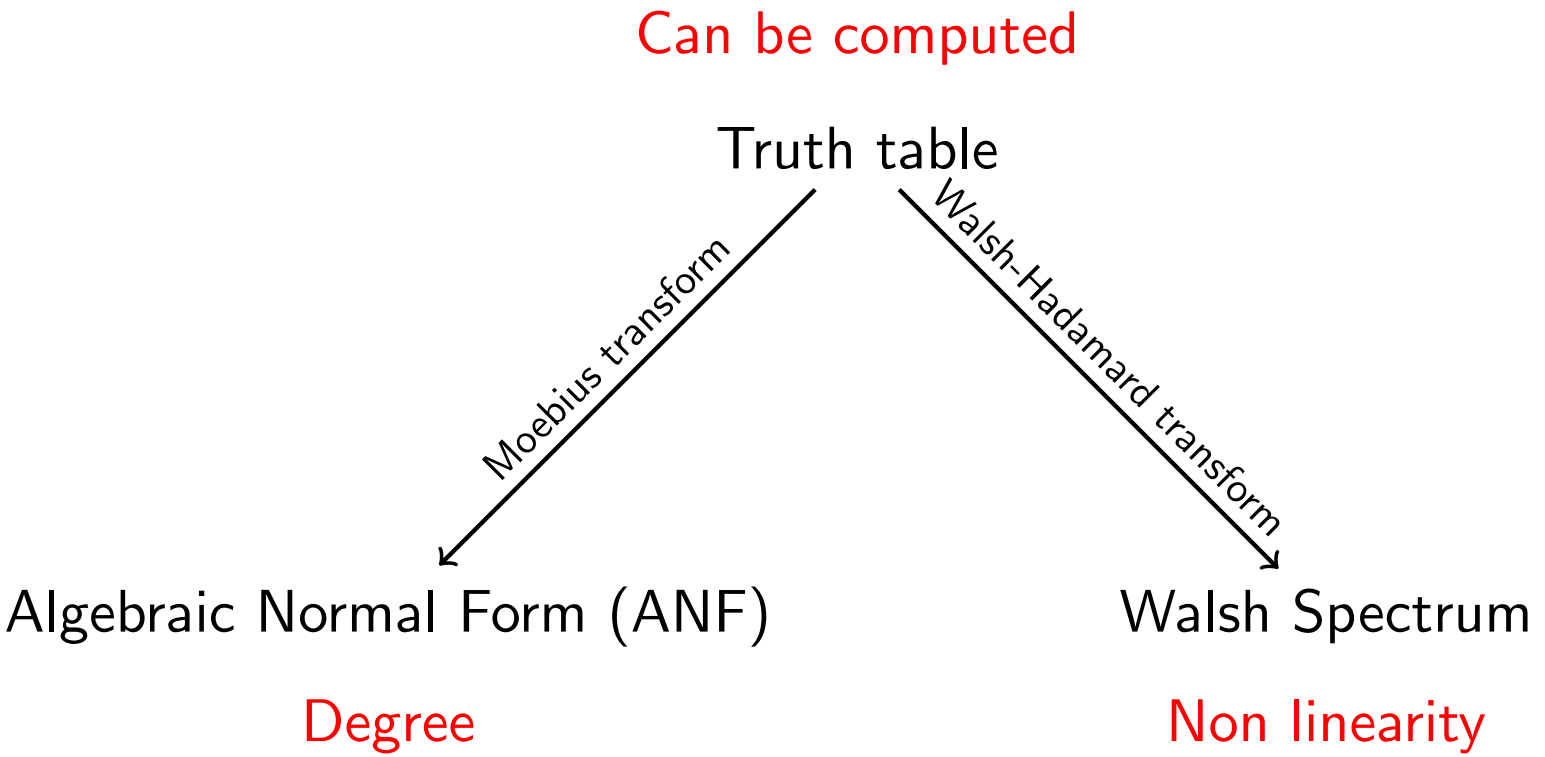


We study the inner function $g : \mathbb{F}_2^n \longmapsto \mathbb{F}_2^4$.



Security analysis details over $\mathbb{F}_2$

We study the inner function $g : \mathbb{F}_2^n \longmapsto \mathbb{F}_2^4$.



Security analysis - Truth table



Binary table we get from computing the function for every input.
For a n variable Boolean function, the truth table has a length of 2^n.

Security analysis - Truth table



Binary table we get from computing the function for every input.
For a n variable Boolean function, the truth table has a length of 2^n.

Example:

$$f : (x_0, x_1) \longmapsto x_0 x_1 + 1$$

$$\begin{aligned} TT(f) &= [f(0,0), f(0,1), f(1,0), f(1,1)] \\ &= [1,1,1,0] \end{aligned}$$

Security analysis - ANF (reminder)



The ANF is a way of writing a BF as a sum of monomials.
For n-variable function, there are 2^n monomials. The ANF is a binary table of length 2^n (1 if the monomial appears in the ANF, 0 else).

Security analysis - ANF (reminder)



The ANF is a way of writing a BF as a sum of monomials.
For n-variable function, there are 2^n monomials. The ANF is a binary table of length 2^n (1 if the monomial appears in the ANF, 0 else).

Example:

$$f : (x_0, x_1) \longmapsto x_0 x_1 + 1$$

$$ANF(f) = [1, 0, 0, 1]$$

Security analysis - ANF optimization



Computing the ANF for all the subfonctions is expensive. We sort the monomials by degree so that we can only partially compute the guesses.

Security analysis - ANF optimization



Computing the ANF for all the subfonctions is expensive. We sort the monomials by degree so that we can only partially compute the guesses.

Example:

$$f : (x_0, x_1, x_2) \longmapsto x_0x_1x_2 + x_0x_2 + x_0 + x_1 + 1$$

$$\begin{aligned} ANF(f) &= [1, 1, 1, 0, 0, 1, 0, 1] \\ &= [0, 1, 2, 5, 7] \\ &= [[7], [5], [1, 2], [0]] \end{aligned}$$

Then, we find the highest degree monomials that is not canceled by the guess.

Attack complexities



- Degree $\rightarrow$ complexity against algebraic attacks (somewhere between naive Gaussian reduction and Fast Algebraic Attacks)
- Nonlinearity $\rightarrow$ complexity against correlation-like attacks
- Number of monomials $\rightarrow$ complexity against [GHBJR23] attack

Elisabeth-b4



Parameter increase:

- More parallel inner functions: ~~12~~ $\rightarrow$ 14
- Inner function takes more variables: ~~5~~ $\rightarrow$ 7

Elisabeth-b4



Parameter increase:

- More parallel inner functions: ~~12~~ $\rightarrow$ 14
- Inner function takes more variables: ~~5~~ $\rightarrow$ 7

With the inner function mixing more chunks, we get enough monomials to prevent the linearization attack.

Gabriel-4



Gabriel-4's direct sum mixes:

- 8 inner functions g_L from Elisabeth-4
- 10 inner functions g_R from Elisabeth-b4

Gabriel-4

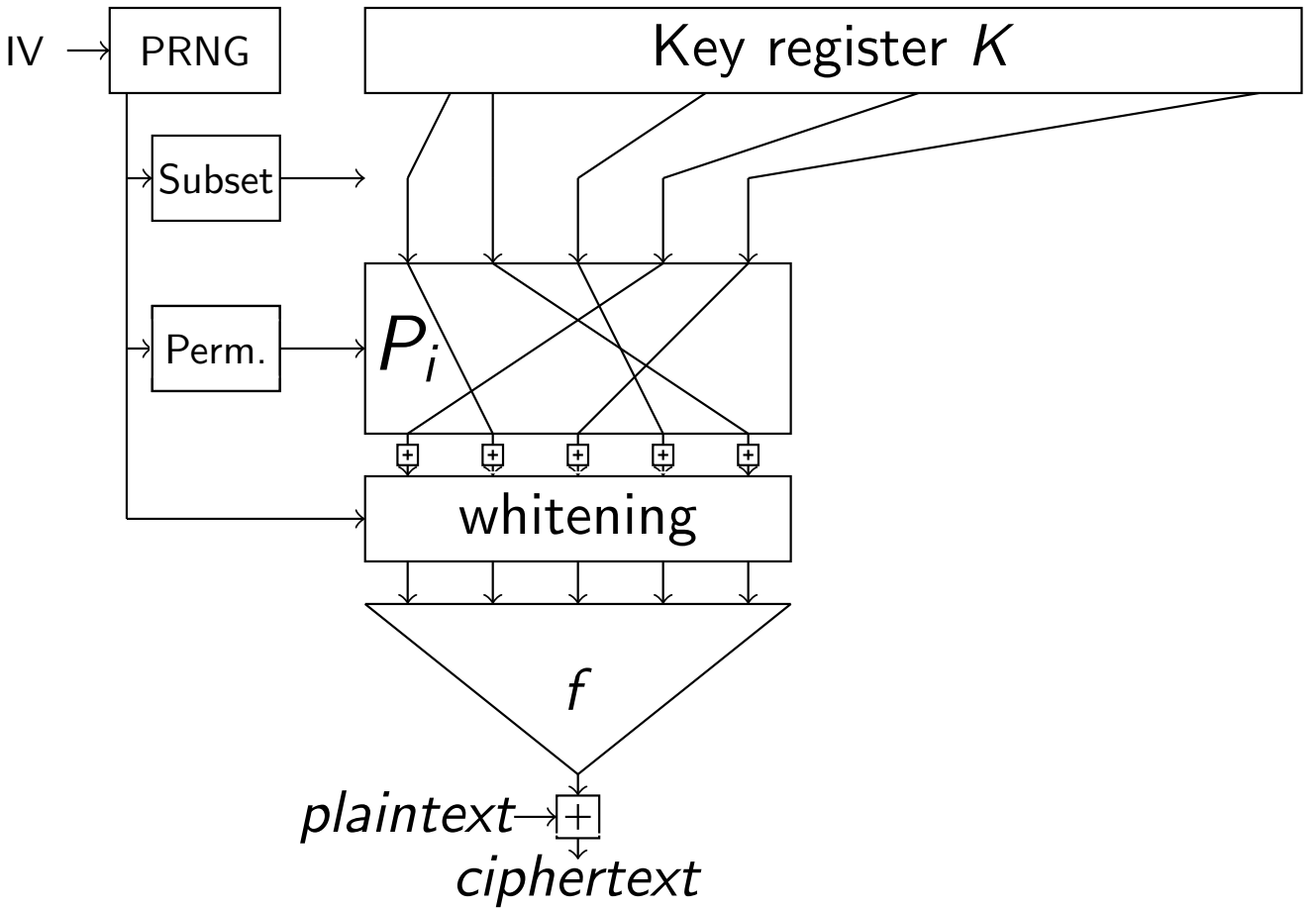


Gabriel-4's direct sum mixes:

- 8 inner functions g_L from Elisabeth-4
- 10 inner functions g_R from Elisabeth-b4

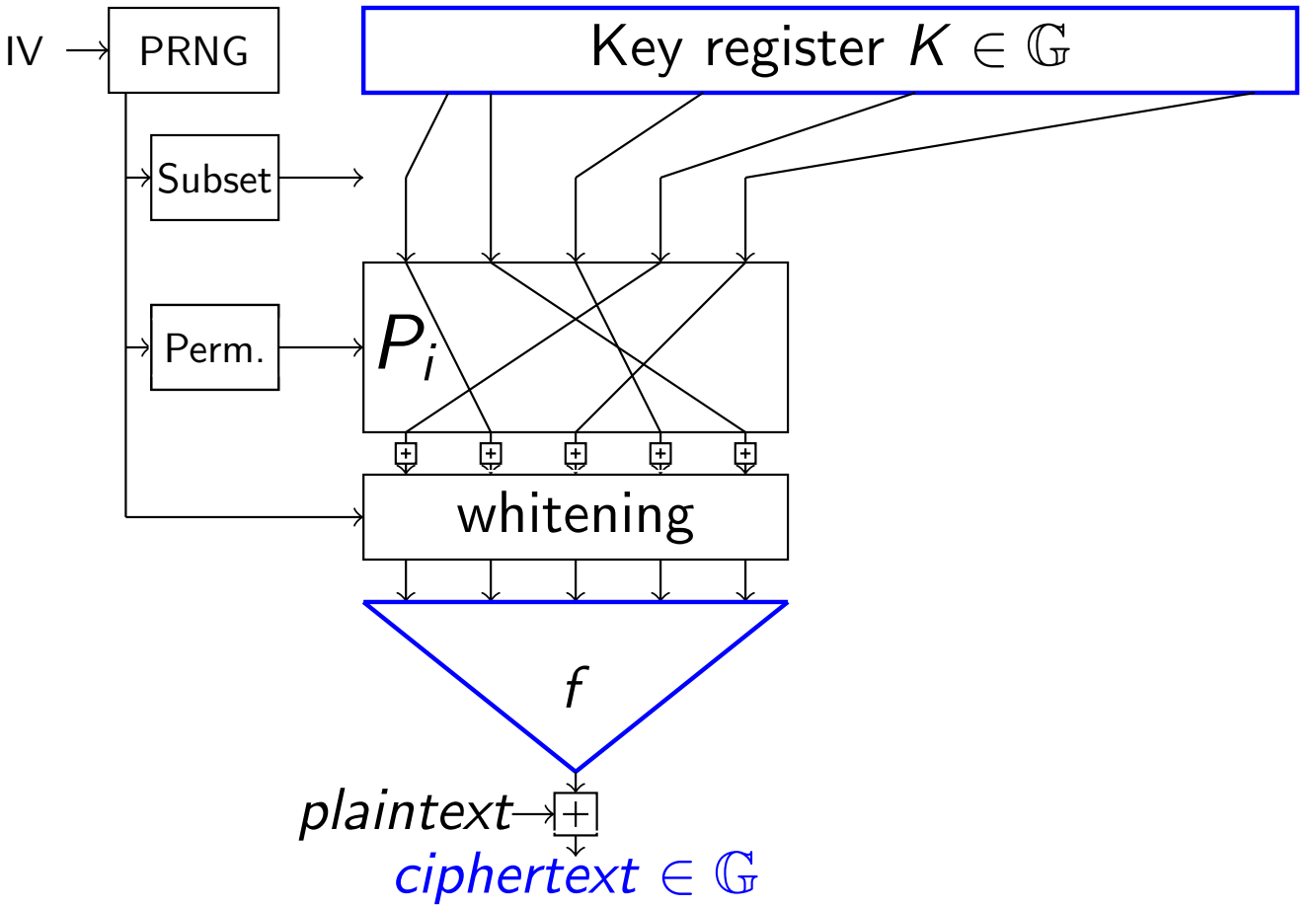
Overall less expensive than Elisabeth-b4, and we reach enough monomial in our final keystream.

Mixed Filter Permutator



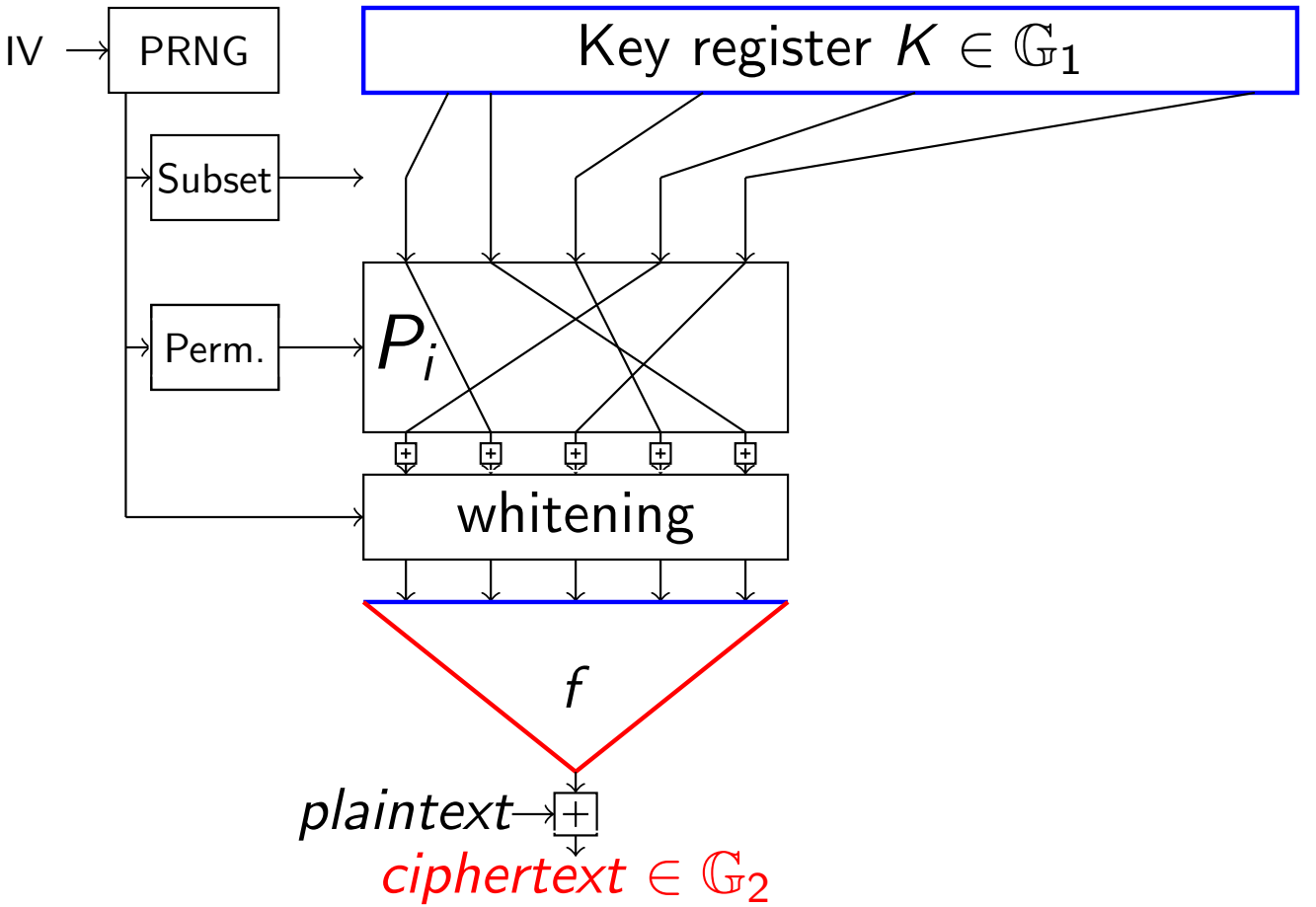
The Group Filter Permutator design

Mixed Filter Permutator



The Group Filter Permutator design

Mixed Filter Permutator



The Mixed Filter Permutator design

Margrethe-18-4



- $\mathbb{G}_1 = \mathbb{F}_2$, $\mathbb{G}_2 = \mathbb{Z}_{16}$
- The filter is the direct sum of **14** identical **18**-variable inner functions
- The inner function is a **18-to-4** look-up table

Margrethe-18-4



- $\mathbb{G}_1 = \mathbb{F}_2$, $\mathbb{G}_2 = \mathbb{Z}_{16}$
- The filter is the direct sum of **14** identical **18**-variable inner functions
- The inner function is a **18-to-4** look-up table

Pros: key in $\mathbb{F}_2$ = no more chunks, no more sparse system

Cons: homomorphic implementation left to do *but* many papers going in this direction

Conclusion



- Many different schemes due to the homomorphic constraints, but the same underlying security analysis
- Margrethe homomorphic evaluation left to implement

Conclusion



- Many different schemes due to the homomorphic constraints, but the same underlying security analysis
- Margrethe homomorphic evaluation left to implement

Thank You!

Algorithm 1: Elisabeth-b4 7-to-1 function.

] **input :** $(x_1, x_2, x_3, x_4, x_5, x_6, x_7) \in \mathbb{Z}_{16}^7$
output: $z \in \mathbb{Z}_{16}$
begin
  for i in range(3) **do**
    $x_{2i+2} = x_{2i+2} + x_{2i+1}$
  for i in range(6) **do**
    `// First round of NLUTs`
    $y_{i+1} = S_{i+1}(x_{i+1})$
  for i in range(3) **do**
    $z_{2i+1} = y_{2i \mod 6} + y_{2i+1}$
    $z_{2i+2} = y_{2i+5 \mod 6} + y_{2i+2}$
  for i in range(6) **do**
    $z_{i+1} = z_{i+1} + x_{i+3 \mod 6}$
    `// Second round of NLUTs`
    $z_{i+1} = S_{i+7}(z_{i+1})$
  for i in range(2) **do**
    $t_{3i+1} = z_{3i+1} + z_{3i+2} + z_{3i+3}$
    $t_{3i+2} = z_{3i+2} + z_{3i+4 \mod 6}$
    $t_{3i+3} = z_{3i+3} + z_{3i+4 \mod 6} + y_{3i+1}$
  $t_1 = t_1 + x_6$
  $t_2 = t_2 + x_5$
  $t_3 = t_3 + x_4$
  $t_4 = t_4 + x_2$
  $t_5 = t_5 + x_1$
  $t_6 = t_6 + x_3$
  $z = x_7$
  for i in range(6) **do**
    `// Third round of NLUTs`
    $u_{i+1} = S_{i+13}(t_{i+1})$
    $z = z + u_{i+1}$
  return z